

Programmation Linéaire & Python

**maximiser le nombre total de groupes
répartis de manière optimale**

3^{eme} année Ingénierie Informatique et Réseaux

Réalisé par :

Aghamri Badreddine - Talami Abderrahman
Malainine Mohamed

Encadré par :

Abdelati REHA & Yassine SAFSOUF :

Année Universitaire 2025-2026

Table des matières

INTRODUCTION	1
CHAPITRE 1 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE GRAPHIQUE	2
I.1. Introduction.....	2
I.2. Contexte	2
I.3. Programme Linéaire	3
I.4. Résolution avec la méthode graphique.....	4
I.5. Validation avec le solveur d'Excel.....	7
I.6. Conclusion	7
CHAPITRE 2 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE SIMPLEXE / DUALITE	9
II.1. Introduction.....	9
II.2. Contexte	9
II.3. Programme Linéaire	10
II.4. Résolution avec la méthode Simplexe / Dualité.....	10
II.5. Validation avec le solveur d'Excel.....	13
II.6. Conclusion	13
CHAPITRE 3 : CONCEPTION INFORMATIQUE.....	15
III.1. Introduction.....	15
III.2. xxxxxx.....	15
III.3. yyyyy	17
III.4. Conclusion	41
CONCLUSION GENERALE ET PERSPECTIVES	41
ANNEXES.....	43
Annexe 1 : Listing du Programme xxxxx	Erreur ! Signet non défini.
Annexe 2: Listing du Programme yyyy	Erreur ! Signet non défini.
REFERENCES.....	46

INTRODUCTION

Ce projet s'inscrit dans un contexte marqué par un besoin croissant d'optimisation, de modernisation et d'amélioration des performances au sein de l'organisation, confrontée à des contraintes techniques, organisationnelles et concurrentielles de plus en plus exigeantes. Face à l'évolution des technologies, à l'augmentation des attentes des utilisateurs et à la nécessité d'améliorer la qualité, la fiabilité et la rapidité des services proposés, il devient indispensable de mettre en place une solution structurée, efficace et durable. L'objectif principal du projet est donc de concevoir et de déployer une solution adaptée permettant de répondre précisément aux besoins identifiés, tout en respectant les contraintes de coût, de délai et de qualité définies dans le cahier des charges. Plus spécifiquement, le projet vise à analyser la situation existante, identifier les dysfonctionnements ou limites actuelles, proposer une solution technique et/ou organisationnelle optimisée, améliorer la performance globale du système, renforcer la satisfaction des utilisateurs finaux et assurer une meilleure gestion des ressources. À travers cette démarche, l'organisation ambitionne également de garantir la pérennité de ses activités, d'augmenter sa compétitivité et de s'inscrire dans une dynamique d'amélioration continue. Le présent projet porte sur la conception et la modélisation d'un système d'optimisation des ressources académiques pour les classes de 3ème année en Ingénierie Informatique à l'EMSI Gueliz. L'enjeu central est de résoudre le problème complexe de l'ordonnancement des cours (Timetabling) en utilisant les principes de la programmation linéaire. L'objectif principal est de maximiser l'efficacité de l'emploi du temps en assurant une répartition fluide des séances sur les trois niveaux dédiés du campus (Étages 1, 2 et 3), tout en minimisant les périodes d'inactivité pour les étudiants et en optimisant l'usage des salles de classe.

Pour atteindre cet objectif, le modèle intègre une série de contraintes critiques garantissant la viabilité de la solution. Parmi elles, les contraintes d'unicité empêchent tout conflit d'horaire pour les professeurs et les groupes d'étudiants, tandis que les contraintes de capacité veillent à ce que chaque groupe soit affecté à une salle adaptée à son effectif. Une attention particulière est portée à la dimension géographique du projet : en limitant les variables aux étages 1, 2 et 3, le modèle cherche à réduire les déplacements inutiles et à saturer intelligemment les espaces disponibles avant de solliciter d'autres ressources.

Sur le plan technique, ce problème est formulé comme un programme linéaire en variables mixtes. La fonction objectif vise à maximiser un score de "confort" et d'efficacité, pondéré par le respect des volumes horaires pédagogiques. Ce projet ne se contente pas d'une approche théorique ; il utilise des outils de résolution performants comme la bibliothèque PuLP ou Google OR-Tools sous Python pour transformer des contraintes logistiques réelles en une solution mathématique rigoureuse, offrant ainsi une gestion moderne et automatisée de la vie scolaire à l'EMSI.

CHAPITRE 1 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE GRAPHIQUE

I.1.Introduction

Dans les établissements d'enseignement supérieur, la gestion et l'optimisation de l'utilisation des salles de cours représentent un enjeu important afin d'assurer une organisation efficace des emplois du temps et une utilisation rationnelle des ressources disponibles. Une mauvaise répartition des groupes peut entraîner des conflits d'horaires, une surcharge de certaines salles ou au contraire une sous-utilisation d'autres espaces.

Dans ce contexte, ce projet s'intéresse à l'optimisation de l'utilisation des salles destinées aux étudiants de troisième année Génie Informatique au sein de l'EMSI Guéliz. Le bâtiment étudié est composé de trois étages, chacun disposant de cinq salles de cours, tandis que la promotion comprend dix groupes d'étudiants nécessitant chacun vingt-et-une heures de cours par semaine.

Afin de répondre à cette problématique, nous faisons appel à la programmation linéaire, une méthode mathématique permettant de modéliser et d'optimiser l'utilisation de ressources limitées sous un ensemble de contraintes. Dans ce chapitre, nous allons d'abord formuler le problème sous forme d'un modèle mathématique comprenant une fonction objectif et des contraintes. Ensuite, nous appliquerons la méthode graphique pour analyser les solutions possibles et déterminer la solution optimale permettant une meilleure répartition des groupes dans les salles disponibles.

I.2.Contexte

Dans le cadre de l'organisation pédagogique au sein de l'École Marocaine des Sciences de l'Ingénieur (EMSI), et plus précisément au campus de Guéliz, la gestion des salles de cours constitue un défi important. En effet, l'augmentation du nombre d'étudiants et de groupes nécessite une planification efficace afin d'assurer le bon déroulement des enseignements tout en optimisant l'utilisation des ressources disponibles.

Le présent projet concerne les étudiants de troisième année en Génie Informatique, répartis en dix groupes. Chaque groupe nécessite un volume horaire hebdomadaire de vingt-et-une heures de cours. Par ailleurs, le bâtiment étudié est composé de trois étages, chacun disposant de cinq salles, soit un total de quinze salles disponibles.

Dans ce contexte, il devient essentiel de trouver une répartition optimale des groupes dans les différentes salles et sur les différents étages, tout en respectant les contraintes liées à la capacité des salles et au volume horaire. L'objectif est d'éviter les conflits d'occupation, de réduire les périodes d'inactivité des salles et d'assurer une utilisation équilibrée des infrastructures.

Ainsi, ce problème réel peut être modélisé sous forme d'un problème d'optimisation en programmation linéaire, permettant de déterminer la meilleure affectation possible des groupes aux salles en fonction des contraintes existantes.

I.3. Programme Linéaire

Dans le cadre du programme linéaire modélisant l'optimisation des salles pour les groupes de 3^e année Génie Informatique à l'EMSI Guéliz, nous définissons les variables x et y comme représentant respectivement le nombre de groupes affectés au premier et au deuxième étage. Le nombre de groupes au troisième étage, z , est déterminé automatiquement par la relation $z = 10 - x - y$, de manière à ce que tous les groupes soient pris en compte. La fonction objectif choisie consiste à maximiser le nombre total de groupes répartis de manière optimale dans les salles, exprimée par $Z = x + y$.

Contrainte 1 :

$$x + y \leq 10$$

Cette contrainte représente le nombre total de groupes disponibles. En effet, il existe uniquement 10 groupes de 3^e année Génie Informatique à l'EMSI. Ainsi, la somme des groupes affectés au premier et au deuxième étage ne doit pas dépasser ce total.

Contrainte 2 :

$$21x \leq 175$$

Chaque groupe nécessite 21 heures de cours par semaine. Cette contrainte traduit la capacité maximale du 1^{er} étage, limitée à 175 heures par semaine (5 salles × 35 heures). Le total des heures utilisées par les groupes au premier étage ne doit donc pas dépasser cette capacité.

Contrainte 3 :

$$21y \leq 175$$

De manière similaire, cette contrainte concerne le 2^e étage. Elle garantit que le nombre total d'heures occupées par les groupes affectés à cet étage ne dépasse pas la capacité maximale des salles disponibles.

Contrainte 4 :

$$x - y \leq 4$$

Cette contrainte assure une répartition équilibrée des groupes entre les étages, évitant qu'un étage soit surchargé par rapport à l'autre.

Contraintes de positivité :

$$x \geq 0, y \geq 0$$

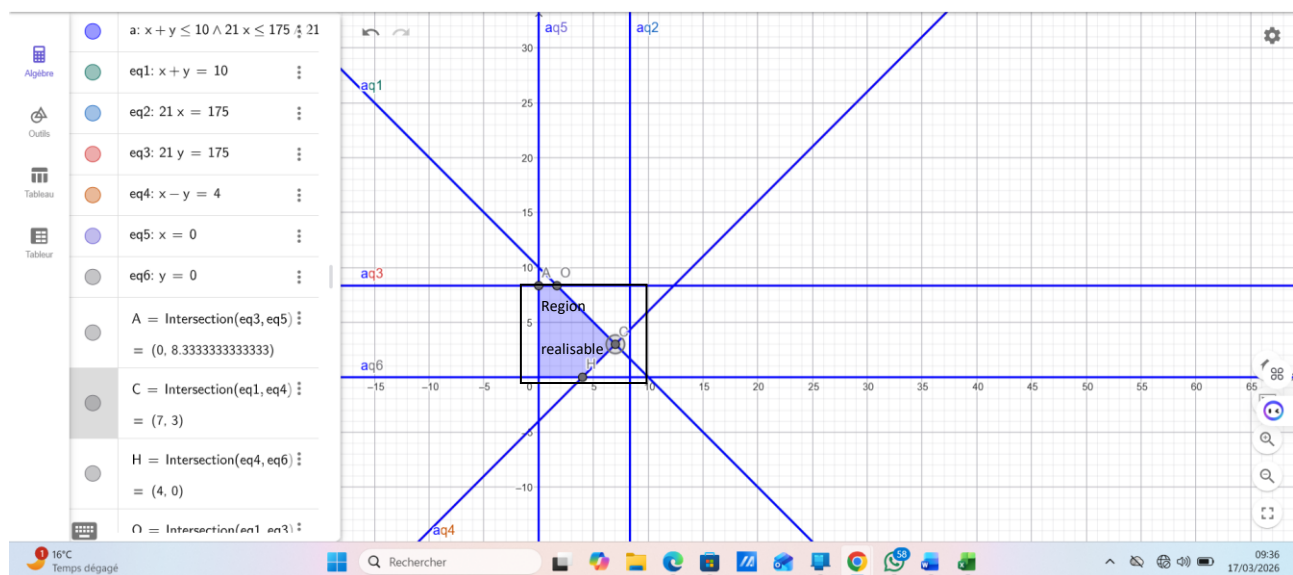
Ces contraintes garantissent que le nombre de groupes reste cohérent avec la réalité, c'est-à-dire non négatif.

À propos du 3^e étage :

Le nombre de groupes affectés au troisième étage, z , est déterminé automatiquement par la relation $z = 10 - x - y$, ce qui permet de prendre en compte tous les groupes de manière cohérente tout en conservant le modèle à deux variables pour la résolution graphique.

1.4. Résolution avec la méthode graphique

Après avoir tracé les différentes régions associées aux contraintes, la région réalisable du programme linéaire est représentée dans la figure ci-après.



 Algèbre		$a: x + y \leq 10 \wedge 21x \leq 175 \wedge 21$	
		eq1: $x + y = 10$	⋮
 Outils		eq2: $21x = 175$	⋮
		eq3: $21y = 175$	⋮
 Tableau		eq4: $x - y = 4$	⋮
		eq5: $x = 0$	⋮
 Tableur		eq6: $y = 0$	⋮
		$A = \text{Intersection}(\text{eq3}, \text{eq5})$ $= (0, 8.33333333333333)$	⋮
		$C = \text{Intersection}(\text{eq1}, \text{eq4})$ $= (7, 3)$	⋮
		$H = \text{Intersection}(\text{eq4}, \text{eq6})$ $= (4, 0)$	⋮
		$O = \text{Intersection}(\text{eq1}, \text{eq3})$	⋮

		eq3: $21y = 175$	⋮
		eq4: $x - y = 4$	⋮
 Algèbre		eq5: $x = 0$	⋮
 Outils		eq6: $y = 0$	⋮
 Tableau		$A = \text{Intersection}(\text{eq3}, \text{eq5})$ $= (0, 8.33333333333333)$	⋮
 Tableur		$C = \text{Intersection}(\text{eq1}, \text{eq4})$ $= (7, 3)$	⋮
		$H = \text{Intersection}(\text{eq4}, \text{eq6})$ $= (4, 0)$	⋮
		$O = \text{Intersection}(\text{eq1}, \text{eq3})$ $= (1.66666666666667, 8.33333333333333)$	⋮

	A	B	C	D	E
1	point	x1	x2	Z	
2	A	0	8,33	8,33	
3	C	7	3	10	maximum
4	H	4	0	4	
5	O	1,67	8,33	10	
6					
7					

Après analyse des différents sommets de la région réalisable, on en déduit que le point C correspond à la solution optimale.

I.5. Validation avec le solveur d'Excel

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	x	5																
2	y	5																
3	z	10																
4	c1	10	inférieur	10														
5	c2	105	inférieur	175														
6	c3	105	inférieur	175														
7	c4	0	inférieur	175														
8	c5	5	supérieur	0														
9	c6	5	supérieur	0														
10																		
11																		
12																		
13																		
14																		
15																		
16																		
17																		
18																		
19																		
20																		
21																		
22																		
23																		
24																		
25																		
26																		

La résolution du programme linéaire, effectuée à l'aide de la méthode graphique et du solveur Excel, a permis de déterminer les solutions optimales du problème d'affectation des groupes aux étages. Les résultats obtenus montrent qu'il existe plusieurs solutions optimales, notamment $((x = 5, y = 5))$ et $((x = 7, y = 3))$, qui donnent toutes la même valeur maximale de la fonction objectif ($Z = 10$). Cette situation s'explique par le fait que la fonction objectif est parallèle à une contrainte, ce qui engendre une infinité de solutions optimales situées sur un segment de la région réalisable.

Parmi ces solutions, la répartition $((5,5))$ apparaît comme la plus équilibrée, avec 5 groupes au premier étage et 5 groupes au deuxième étage, tandis que le troisième étage n'est pas utilisé. Cette solution permet une utilisation efficace et homogène des ressources disponibles, tout en respectant l'ensemble des contraintes du modèle. Ainsi, le problème d'optimisation a été résolu de manière satisfaisante, en fournissant une solution réaliste et applicable dans le contexte étudié.

I.6. Conclusion

Au terme de ce projet, nous avons étudié un problème réel d'optimisation portant sur la gestion et la répartition des salles pour les étudiants de troisième année Génie Informatique à l'EMSI Guéliz. Face à un nombre limité de ressources (salles disponibles et volume horaire), il était nécessaire de mettre en place une méthode efficace permettant d'assurer une organisation optimale des groupes.

Pour cela, nous avons utilisé la programmation linéaire afin de modéliser le problème en définissant des variables de décision, une fonction objectif et un ensemble de contraintes traduisant les conditions réelles. Cette modélisation a ensuite été exploitée à l'aide de la méthode graphique ainsi que du solveur Excel, permettant de déterminer les solutions optimales.

Les résultats obtenus ont montré qu'il existe plusieurs solutions optimales donnant la même valeur maximale de la fonction objectif. Parmi celles-ci, une répartition équilibrée des groupes entre les étages s'est révélée particulièrement pertinente, garantissant une utilisation efficace des salles tout en respectant les contraintes imposées.

Ce projet met ainsi en évidence l'importance des outils d'optimisation dans la prise de décision et la gestion des ressources. Il illustre également comment un problème concret peut être traduit en modèle mathématique puis résolu de manière rigoureuse, offrant des solutions applicables dans un contexte réel.

CHAPITRE 2 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE SIMPLEXE / DUALITE

II.1. Introduction

Dans ce deuxième chapitre, nous abordons une approche plus générale et plus performante pour la résolution des problèmes d'optimisation linéaire, à savoir la méthode du simplexe ainsi que la notion de dualité. Contrairement à la méthode graphique, limitée aux problèmes à deux variables, la méthode du simplexe permet de traiter des modèles comportant un nombre plus important de variables et de contraintes.

L'objectif de ce chapitre est de reformuler le problème étudié précédemment sous une forme standard adaptée à l'application de cette méthode, puis de le résoudre à l'aide de tableaux itératifs. Cette démarche permettra de déterminer la solution optimale tout en mettant en évidence les mécanismes de calcul et d'évolution des variables au cours des différentes itérations.

II.2. Contexte

Dans le cadre de l'optimisation de l'affectation des salles pour les groupes de 3^e année Génie Informatique à l'EMSI Guéliz, il est nécessaire de disposer d'une méthode de résolution efficace et structurée. En effet, le problème consiste à répartir un ensemble de groupes entre plusieurs étages, tout en respectant des contraintes liées à la capacité horaire des salles ainsi qu'à l'équilibre de la répartition.

Bien que la méthode graphique ait permis d'obtenir une première solution, elle reste limitée en termes de généralisation. C'est pourquoi l'utilisation de la méthode du simplexe s'impose comme une solution plus adaptée, permettant de valider les résultats obtenus précédemment et d'assurer une résolution systématique du problème. Cette approche offre également une meilleure compréhension de la structure du programme linéaire et de l'impact des contraintes sur la solution optimale.

II.3. Programme Linéaire

Le problème est modélisé sous la forme d'un programme linéaire visant à maximiser la fonction objectif correspondant au nombre total de groupes répartis dans les étages. Les variables de décision x et y représentent respectivement le nombre de groupes affectés au premier et au deuxième étage, tandis que le nombre de groupes affectés au troisième étage est donné par la relation $z = 10 - x - y$. La fonction objectif s'écrit donc comme la maximisation de $Z = x + y$.

Afin d'appliquer la méthode du simplexe, le modèle est transformé en une forme standard en introduisant des variables d'écart pour convertir les inégalités en égalités. Ainsi, chaque contrainte est associée à une variable d'écart (notées $e1$, $e2$, $e3$ et $e4$), représentant la capacité non utilisée ou la marge disponible. Les contraintes deviennent alors :

$$x + y + e1 = 10$$

$$21x + e2 = 175$$

$$21y + e3 = 175$$

$$x - y + e4 = 4$$

avec les conditions de non-négativité :

$$x \geq 0, y \geq 0, e1 \geq 0, e2 \geq 0, e3 \geq 0, e4 \geq 0.$$

Cette transformation permet de construire le tableau initial du simplexe et de procéder aux différentes itérations menant à la solution optimale. Les variables d'écart jouent un rôle essentiel dans cette méthode, car elles constituent la base initiale du système et facilitent l'évolution progressive vers la solution optimale.

II.4. Résolution avec la méthode Simplexe / Dualité

La résolution du programme linéaire est effectuée à l'aide de la méthode du simplexe, une technique algorithmique permettant de déterminer la solution optimale d'un problème d'optimisation linéaire comportant plusieurs contraintes. Contrairement à la méthode graphique, limitée à deux variables, la méthode du simplexe permet de traiter le problème de manière plus générale et plus rigoureuse.

Dans ce cadre, le modèle étudié reprend les mêmes hypothèses que celles du chapitre précédent. Les variables de décision x et y représentent respectivement le nombre de groupes affectés au premier et au deuxième étage, tandis que le nombre de groupes au troisième étage est donné par la relation :

$$z = 10 - x - y$$

La fonction objectif consiste à maximiser le nombre total de groupes affectés :

$$Z = x + y$$

Sous les contraintes suivantes :

$$x + y \leq 10$$

$$21x \leq 175$$

$$21y \leq 175$$

$$x - y \leq 4$$

$$x \geq 0, y \geq 0$$

Afin d'appliquer la méthode du simplexe, ces inéquations sont transformées en équations en introduisant des variables d'écart, ce qui permet de mettre le modèle sous forme standard. Un tableau initial du simplexe est ensuite construit en intégrant la fonction objectif ainsi que l'ensemble des contraintes.

La méthode du simplexe repose sur un processus itératif. À chaque étape, une variable entrante est choisie en fonction des coefficients de la fonction objectif (coefficient le plus négatif), tandis que la variable sortante est déterminée à l'aide du test du rapport minimum. Ces opérations permettent de passer d'une solution de base à une autre tout en améliorant progressivement la valeur de la fonction objectif.

Après plusieurs itérations, la méthode converge vers une solution optimale. Les résultats obtenus montrent qu'il existe plusieurs solutions optimales donnant la même valeur maximale de la fonction objectif :

$$Z = 10$$

Parmi ces solutions, on peut citer :

$$(x = 5, y = 5) \text{ et } (x = 7, y = 3)$$

Ces solutions correspondent à des points situés sur un segment de la région réalisable, ce qui s'explique par le fait que la fonction objectif est parallèle à une contrainte active. Ainsi, il existe une infinité de solutions optimales.

Du point de vue pratique, la solution $(x = 5, y = 5)$ apparaît comme la plus équilibrée, car elle répartit les groupes de manière homogène entre les deux étages, tout en respectant l'ensemble des contraintes.

Par ailleurs, la notion de dualité permet d'apporter une interprétation complémentaire du problème. En effet, chaque contrainte du problème peut être associée à une valeur économique représentant le coût ou l'importance de la ressource correspondante (par exemple, les heures disponibles par étage). Cette approche permet de mieux comprendre l'impact des contraintes sur la solution optimale et d'analyser la sensibilité du modèle.

Ainsi, la méthode du simplexe permet non seulement de déterminer la solution optimale de manière rigoureuse, mais également d'offrir une analyse approfondie du problème à travers le concept de dualité.

1. Tableau initiale :

	A	B	C	D	E	F	G	H	I	J	
1	Z	x	y	z	e1	e2	e3	e4	e5	res	
2		1	-1	-1	-1	0	0	0	0	0	0
3		0	1	1	1	1	0	0	0	0	10
4		0	21	0	0	0	1	0	0	0	175
5		0	0	21	0	0	0	1	0	0	175
6		0	0	0	21	0	0	0	1	0	175
7		0	1	-1	0	0	0	0	0	1	4
8											
9											

Le tableau initial représente la mise en forme standard du programme linéaire permettant l'application de la méthode du simplexe. La fonction objectif $Z = x + y$ est transformée sous forme canonique $Z - x - y = 0$, ce qui explique la présence des coefficients négatifs dans la première ligne. Les contraintes du problème sont ensuite converties en équations à l'aide de variables d'écart afin de transformer toutes les inégalités en égalités exploitables par l'algorithme. Ce tableau constitue la solution de départ, où aucune optimisation n'a encore été réalisée.

Itérations de l'algorithme du Simplexe

12											
13	Z	x	y	z	e1	e2	e3	e4	e5	res	
14		1	0	-1	-1	0	0,05	0	0	0	8,33
15		0	0	1	1	1	-0,05	0	0	0	10-175/21
16		0	1	0	0	0	0,05	0	0	0	175/21
17		0	0	21	0	0	0	1	0	0	175
18		0	0	0	21	0	0	0	1	0	175
19		0	0	-1	0	0	-0,05	0	0	1	4-175/21
20											
21											
22											

Le tableau après pivot correspond à la première étape d'amélioration de la solution initiale. La variable entrante est sélectionnée à partir de la fonction objectif, en choisissant le coefficient le plus négatif. La ligne pivot est déterminée à l'aide du test des rapports entre les ressources disponibles et les coefficients positifs de la colonne pivot, ce qui permet d'identifier la contrainte limitante.

L'élément pivot est ensuite utilisé pour normaliser la ligne correspondante et transformer la variable entrante en variable de base. Des opérations élémentaires sont appliquées aux autres lignes afin d'annuler cette variable dans tout le tableau. Cette étape permet d'améliorer la valeur de la fonction objectif et de rapprocher la solution de l'optimum.

Tableau optimal

24											
25	Z	x	y	z	e1	e2	e3	e4	e5	res	
26		1	0	0	0	1	0,05	0,05	0,05	0	10
27		0	1	0	0	0	0,05	0	0	0	3
28		0	0	1	0	0	0	0,05	0	0	3
29		0	0	0	1	0	0	0	0,05	0	4
30		0	0	0	0	1	0	0	0	0	0
31		0	0	0	0	0	0	0	0	1	1
32											
33											
34											

Le tableau optimal est atteint lorsqu'il n'existe plus de coefficients négatifs dans la ligne de la fonction objectif, ce qui signifie qu'aucune amélioration supplémentaire n'est possible. Les variables de base donnent directement la solution finale du problème, tandis que les variables d'écart indiquent les ressources restantes dans chaque contrainte.

La solution obtenue respecte l'ensemble des contraintes du modèle et maximise la fonction objectif $Z = x + y$. Elle représente donc la répartition optimale des groupes entre les différents étages, garantissant une utilisation efficace des salles disponibles.

II.5. Validation avec le solveur d'Excel

Afin de vérifier la validité des résultats obtenus par la méthode du simplexe, une résolution du problème a été réalisée à l'aide du module Solveur du logiciel Excel. Cette étape permet de confirmer la cohérence du modèle mathématique ainsi que l'exactitude des calculs effectués manuellement.

Pour ce faire, les variables de décision x et y ont été représentées dans des cellules distinctes, tandis que la fonction objectif a été formulée sous la forme d'une expression dépendant de ces variables. Les contraintes du modèle, à savoir $x + y \leq 10$, $21x \leq 175$, $21y \leq 175$ et $x - y \leq 4$, ont été introduites dans le Solveur en respectant leur formulation mathématique. Les conditions de non-négativité ($x \geq 0$, $y \geq 0$) ont également été prises en compte.

Après exécution du Solveur avec l'option de maximisation de la fonction objectif, la solution obtenue est $x = 5$ et $y = 5$, pour une valeur optimale de $Z = 10$. Cette solution diffère de celle obtenue graphiquement et par la méthode du simplexe, qui donnait $x = 7$ et $y = 3$, mais conduit néanmoins à la même valeur optimale.

Cette différence s'explique par le fait que le problème admet plusieurs solutions optimales. En effet, toutes les combinaisons de valeurs situées sur le segment reliant les points (5, 5) et (7, 3) dans la région réalisable donnent la même valeur maximale de la fonction objectif. Ainsi, le Solveur Excel identifie une solution optimale parmi plusieurs possibles, ce qui confirme la validité globale du modèle et des résultats obtenus.

II.6. Conclusion

En conclusion, l'application de la méthode du simplexe et l'étude de la dualité ont permis de valider rigoureusement le modèle d'optimisation de la répartition des groupes à l'EMSI Guéliz. Cette approche purement mathématique, bien que systématique, souligne la complexité des calculs itératifs nécessaires pour atteindre l'optimum, surtout lorsque le nombre de variables et de contraintes augmente.

Cette étape de modélisation constitue un prérequis indispensable à l'automatisation du problème. En effet, la structure standardisée du simplexe (sous forme de matrices et de tableaux) est directement transposable en algorithmes informatiques. La prochaine étape de ce projet consistera à exploiter cette logique mathématique à travers la programmation, notamment en utilisant le langage **Python** et des bibliothèques spécialisées comme **PuLP** ou **Google OR-Tools**. Ces outils permettent de traduire nos variables de décision et nos contraintes en lignes de code afin de résoudre le problème de manière dynamique, rapide et évolutive, offrant ainsi une solution logicielle concrète aux défis logistiques de l'établissement.

CHAPITRE 3 : CONCEPTION INFORMATIQUE

III.1. Introduction

Dans le cadre de ce projet d'optimisation appliqué à la gestion des salles à l'EMSI, une phase de conception informatique a été réalisée afin de transformer le modèle mathématique en une application fonctionnelle. Cette étape permet d'assurer le passage de la théorie à la pratique en automatisant la résolution du problème.

Le choix du langage Python s'est imposé grâce à sa simplicité, sa lisibilité ainsi que la richesse de ses bibliothèques dédiées au calcul scientifique et à la visualisation des données. En particulier, Python permet d'implémenter efficacement les algorithmes d'optimisation, notamment la méthode du simplexe, tout en offrant des outils puissants pour le développement d'interfaces interactives.

L'application a été développée en adoptant une approche de programmation orientée objet (POO), permettant de structurer le code en différentes classes représentant les éléments du problème, tels que le modèle d'optimisation et le solveur. Cette approche garantit une meilleure organisation, une réutilisabilité du code et une maintenance simplifiée.

Par ailleurs, une interface utilisateur moderne a été intégrée afin de faciliter l'interaction avec l'application. Cette interface permet de saisir les paramètres du problème, d'afficher les résultats de manière claire et de visualiser la solution optimale à l'aide de graphiques.

Ainsi, cette conception informatique aboutit à la réalisation d'un outil interactif et performant, permettant d'optimiser la répartition des salles de manière automatisée et efficace.

III.2. Conception orientée objet

Dans le cadre du développement de cette application, la programmation orientée objet a été adoptée afin de structurer le système de manière claire, modulaire et évolutive. Cette approche permet de décomposer le problème global en plusieurs entités indépendantes, chacune étant modélisée par une classe spécifique ayant un rôle bien défini.

La classe *ProblemeOptimisation* représente l'abstraction du problème à résoudre. Elle encapsule les données d'entrée, notamment le nombre total de groupes ainsi que le volume horaire disponible. Cette classe joue un rôle essentiel en isolant les paramètres du problème, ce qui permet de rendre le système flexible et facilement adaptable à différents scénarios.

La classe *Solveur* constitue le cœur logique de l'application. Elle est responsable de la résolution du problème d'optimisation en utilisant des méthodes de programmation linéaire. Elle prend en entrée un objet de type *ProblemeOptimisation* et applique un algorithme basé sur la fonction *linprog* afin de

déterminer la meilleure répartition des groupes sur les différents étages. Cette séparation entre les données (problème) et le traitement (solveur) respecte le principe de responsabilité unique et améliore la lisibilité du code.

La classe *Solution* est utilisée pour encapsuler les résultats obtenus après la résolution. Elle stocke les valeurs optimales des variables de décision ainsi que la valeur de la fonction objectif. Cette classe facilite la manipulation et l'affichage des résultats dans les différentes parties de l'application, notamment dans l'interface utilisateur.

Par ailleurs, la classe *GestionnaireEDT* assure la gestion complète des emplois du temps. Elle intègre des mécanismes de génération et de vérification permettant de garantir l'absence de conflits, que ce soit au niveau des salles ou des groupes. Cette classe s'appuie sur des structures de données adaptées pour contrôler l'occupation des ressources et assurer le respect des contraintes pédagogiques.

Enfin, cette conception orientée objet permet de respecter des principes fondamentaux tels que l'encapsulation, la modularité et la séparation des responsabilités. Elle facilite également la maintenance du code, son évolution future ainsi que la réutilisation de certains composants dans d'autres applications similaires.

Définition des classes (POO principale)

class Solution:

```
def __init__(self, x, y, z, valeur):  
    self.x = x; self.y = y; self.z = z; self.valeur = valeur
```

class ProblemeOptimisation:

```
def __init__(self, total_groupes, heures):  
    self.total_groupes = total_groupes; self.heures = heures
```

class Solveur:

```
def resoudre(self, p):  
    c = [-1, -1]  
    A = [[1, 1], [p.heures, 0], [0, p.heures], [1, -1]]  
    b = [p.total_groupes, 175, 175, 4]  
    result = linprog(c, A_ub=A, b_ub=b, bounds=(0, None))  
    x, y = result.x if result.success else (0, 0)  
    z = max(0, p.total_groupes - x - y)  
    return Solution(x, y, z, x + y)
```

Classe avancée (gestion complexe)

class GestionnaireEDT:

```
def __init__(self):
    self.grille_salle = {}
    self.grille_groupe = {}
    self.edts = {}
    self.conflits_log = []
```

Méthode dans une classe (POO = comportement)

```
def generer_edt_groupe(self, groupe_id: int, salle: str):
def rapport_conflits(self):
```

Création des objets ()

```
p = ProblemeOptimisation(total_groupes, heures)
solver = Solveur()
sol = solver.resoudre(p)
gestionnaire = GestionnaireEDT()
```

Utilisation des objets

```
gestionnaire.generer_edt_groupe(gid, salle)
nb_cs, nb_cg, details = gestionnaire.rapport_conflits()
```

La programmation orientée objet est mise en œuvre à travers la définition de plusieurs classes telles que *ProblemeOptimisation*, *Solveur*, *Solution* et *GestionnaireEDT*. Chaque classe encapsule des données et des méthodes spécifiques, permettant une meilleure organisation du code. Les objets sont ensuite instanciés et utilisés pour exécuter les différentes fonctionnalités de l'application, notamment la résolution du problème d'optimisation et la génération des emplois du temps.

III.3. Interface Utilisateur de l'Application

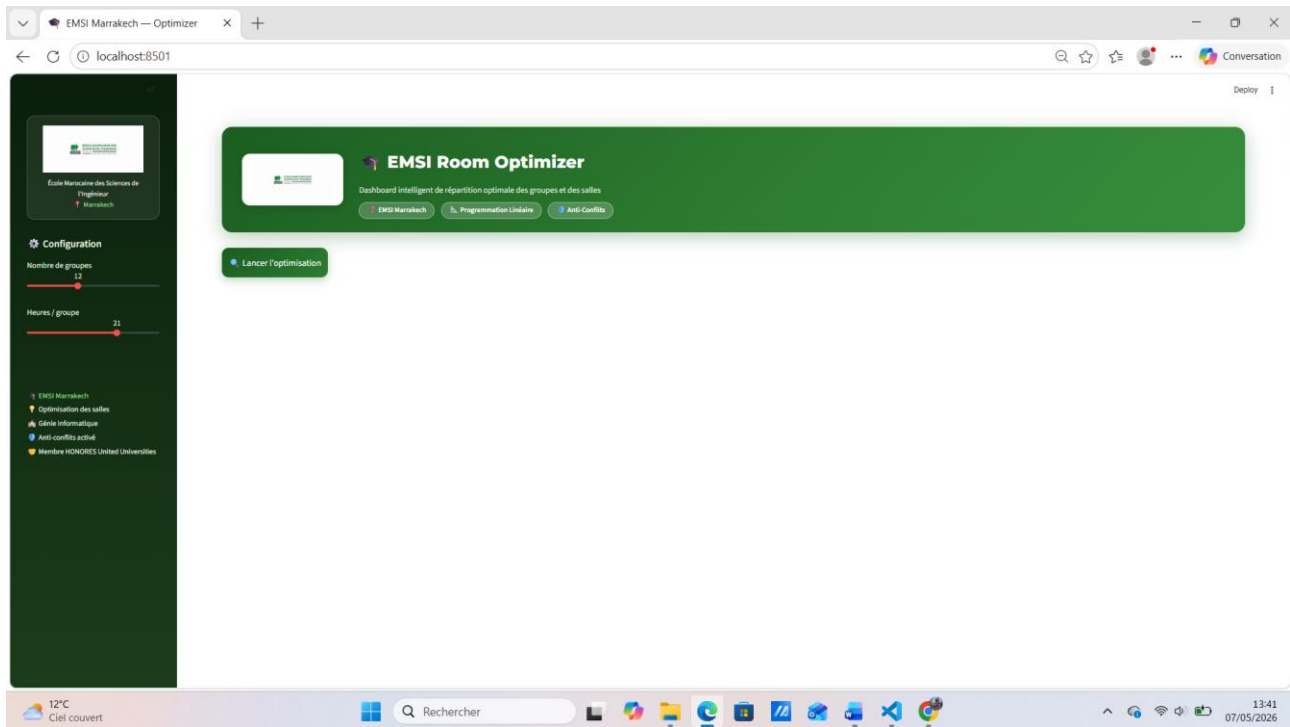
L'interface utilisateur de l'application a été développée à l'aide de la bibliothèque *Streamlit*, qui permet de créer rapidement des applications interactives en Python. Cette interface a été conçue de manière intuitive et ergonomique afin de faciliter l'interaction avec l'utilisateur, même sans connaissances techniques avancées.

La page principale se présente sous forme d'un tableau de bord (dashboard) intégrant plusieurs composants visuels. Une barre latérale permet à l'utilisateur de configurer les paramètres du problème, notamment le nombre de groupes et le nombre d'heures par groupe, à l'aide de curseurs interactifs. Un bouton principal permet ensuite de lancer le processus d'optimisation.

Une fois l'optimisation exécutée, les résultats sont affichés de manière claire à travers plusieurs indicateurs de performance (KPIs), tels que la répartition des groupes par étage et la valeur de la fonction objectif. Ces résultats sont également accompagnés d'une représentation graphique permettant de visualiser la distribution optimale des groupes.

En complément, l'application génère automatiquement les emplois du temps sous forme de tableaux structurés, facilitant leur lecture et leur analyse. Un système de détection des conflits est également intégré, permettant d'identifier rapidement les éventuels problèmes liés à l'affectation des salles ou des créneaux horaires.

Enfin, l'interface adopte un design moderne avec un thème personnalisé inspiré de l'identité visuelle de l'établissement, ce qui améliore l'expérience utilisateur et rend l'application plus professionnelle.





École Marocaine des Sciences de
l'Ingénieur
Marrakech

Configuration

Nombre de groupes

12

Heures / groupe

21

- EMSI Marrakech
- Optimisation des salles
- Génie Informatique
- Anti-conflits activé
- Membre HONORES United Universities



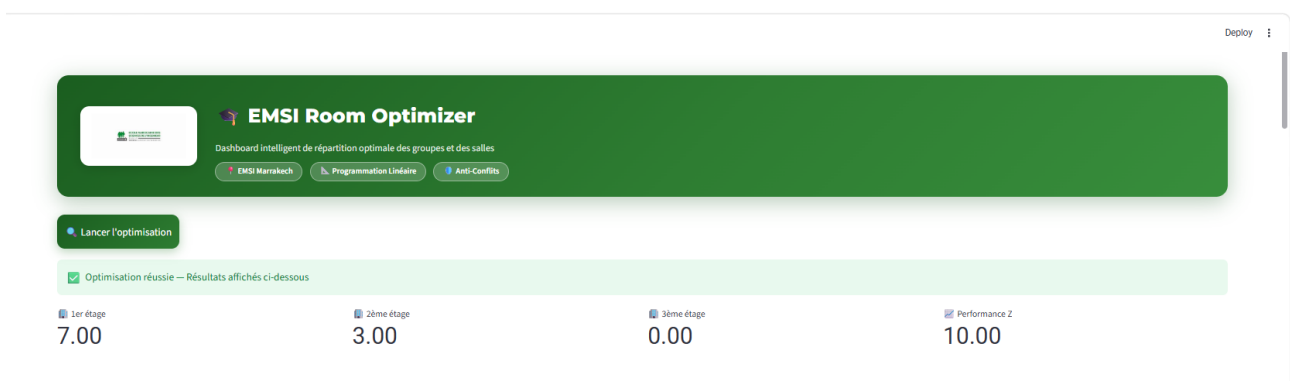
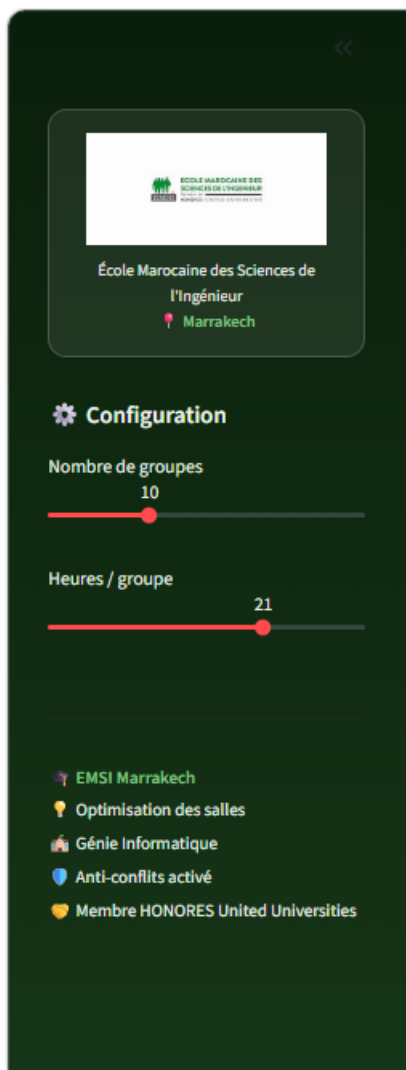
EMSI Room Optimizer

Dashboard intelligent de répartition optimale des groupes et des salles

EMSI Marrakech Programmation Linéaire Anti Conflits

Lancer l'optimisation

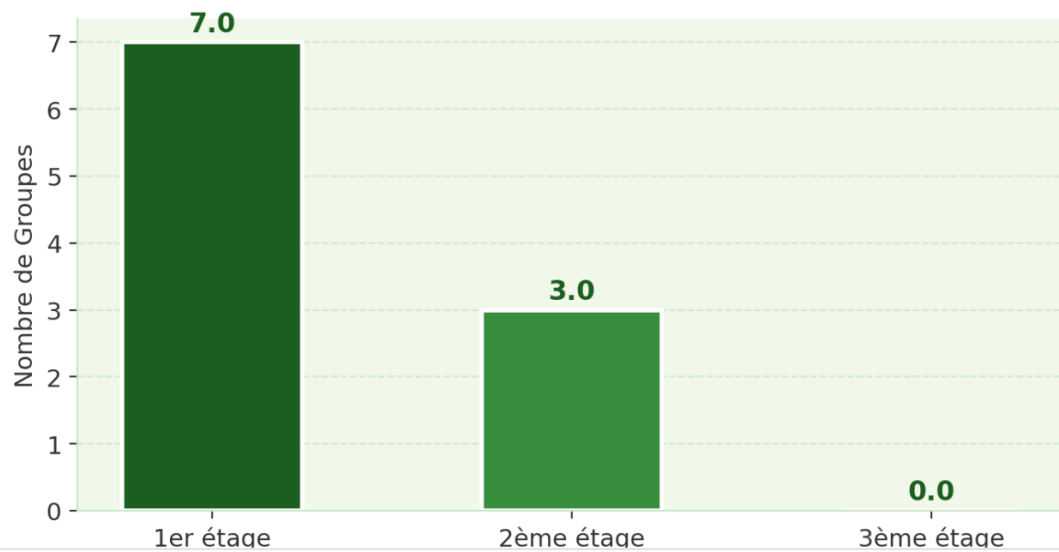
Deploy



Visualisation de la répartition



Distribution optimale des salles — EMSI Marrakech



EMSI Marrakech — Optimizer

localhost:8501

Deploy

0 1er étage 2ème étage 3ème étage 0.0

Analyse automatique

Insight : Le 1er étage est prioritaire.

Taux d'utilisation globale

Utilisation : 100%

Génération des emplois du temps — Détection de conflits

Rapport de Conflits

Conflits de Salle 0 Conflits de Groupe 0 Total Conflits 0

Aucun conflit détecté — Chaque groupe a sa propre salle unique. Tous les emplois du temps sont cohérents.

12°C Ciel couvert 13:44 07/05/2026

EMSI Marrakech — Optimizer

localhost:8501

Conversation

Ecole Marocaine des Sciences de l'Ingénieur

Marrakech

Configuration

Nombre de groupes

10

Heures / groupe

21

EMSI Marrakech

Optimisation des salles

Génie Informatique

Anti-conflits actif

Membre HONORES United Universities

Conflits de Salle

0

Conflits de Groupe

0

Total Conflits

0

Aucun conflit détecté — Chaque groupe a sa propre salle unique. Tous les emplois du temps sont cohérents.

Affectation des salles

Groupe	Étage	Salle
Groupe 1	1er étage	Salle 101
Groupe 2	1er étage	Salle 102
Groupe 3	1er étage	Salle 103
Groupe 4	1er étage	Salle 104
Groupe 5	1er étage	Salle 105
Groupe 6	1er étage	Salle 106
Groupe 7	1er étage	Salle 107
Groupe 8	2ème étage	Salle 108
Groupe 9	2ème étage	Salle 109
Groupe 10	2ème étage	Salle 110

Emplois du temps détaillés

12°C

Ciel couvert

Rechercher

13:46

07/05/2026

EMSI Marrakech — Optimizer

localhost:8501

Conversation

Ecole Marocaine des Sciences de l'Ingénieur

Marrakech

Configuration

Nombre de groupes

10

Heures / groupe

21

EMSI Marrakech

Optimisation des salles

Génie Informatique

Anti-conflits actif

Membre HONORES United Universities

Emplois du temps détaillés

Groupe 1

1er étage | Salle 101

Sans conflit

	Lundi	Mardi	Mercredi	Jeudi	Vendredi
08h30-10h00	Programmation Java	Réseaux Informatiques	Repos	Repos	Programmation Linéaire
10h15-11h45	Développement Projet	Repos	Programmation Java	Repos	Conception OO
12h00-13h30	Repos	English	Repos	Programmation Python	Repos
13h45-15h15	Communication Pro	SQL Server	Repos	Modèles Statistiques	Repos
15h30-17h00	Recherche Scientifique	Repos	Repos	Repos	Modèles Statistiques

Groupe 2

1er étage | Salle 102

Sans conflit

	Lundi	Mardi	Mercredi	Jeudi	Vendredi
08h30-10h00	Repos	Repos	Repos	Modèles Statistiques	Conception OO
10h15-11h45	Recherche Scientifique	Repos	Repos	Programmation Java	Communication Pro
12h00-13h30	Repos	Repos	Repos	Réseaux Informatiques	Repos
13h45-15h15	Repos	Repos	Programmation Java	English	Programmation Python
15h30-17h00	SQL Server	Modèles Statistiques	Repos	Programmation Linéaire	Développement Projet

12°C

Ciel couvert

Rechercher

13:46

07/05/2026

EMSI Marrakech — Optimizer
+ localHost:8501
Conversation



École Marocaine des Sciences de l'Ingénieur
Marrakech

Configuration

Nombre de groupes
10

Heures / groupe
21

- 📍 EMSI Marrakech
- 🔧 Optimisation des salles
- 💡 Génie Informatique
- ⚖️ Anti-conflits activé
- 👤 Membre HONORE'S United Universities

Groupe 3

1er étage | Salle 103 Sans conflit

	Lundi	Mardi	Mercredi	Judi	Vendredi
08h30-10h00	Modèles Statistiques	Repos	Repos	Programmation Python	Communication Pro
10h15-11h45	Modèles Statistiques	Repos	Repos	Repos	Repos
12h00-13h30	Réseaux Informatiques	Développement Projet	Repos	Repos	Recherche Scientifique
13h45-15h15	Repos	Programmation Java	Repos	Programmation Linéaire	Programmation Java
15h30-17h00	Conception OO	Repos	Repos	SQL Server	English

Groupe 4

1er étage | Salle 104 Sans conflit

	Lundi	Mardi	Mercredi	Judi	Vendredi
08h30-10h00	Repos	Modèles Statistiques	SQL Server	Conception OO	Repos
10h15-11h45	English	Repos	Réseaux Informatiques	Programmation Linéaire	Repos
12h00-13h30	Recherche Scientifique	Programmation Python	Repos	Communication Pro	Repos
13h45-15h15	Programmation Java	Repos	Développement Projet	Repos	Repos
15h30-17h00	Modèles Statistiques	Repos	Programmation Java	Repos	Repos

Configuration

- Nombre de groupes: 10
- Heures / groupe: 21
- ESMI Marrakech
- Optimisation des salles
- Génie Informatique
- Anti-conflits actif
- Membre HONORES United Universities

Groupe 7
1er étage | Salle 107 | Sans conflit

	Lundi	Mardi	Mercredi	Jeudi	Vendredi
08h30-10h00	Repos	Conception OO	Repos	Repos	Repos
10h15-11h45	Repos	Repos	Recherche Scientifique	Repos	Repos
12h00-13h30	Repos	Programmation Python	Modèles Statistiques	Repos	Développement Projet
13h45-15h15	Programmation Java	Modèles Statistiques	Communication Pro	Repos	Réseaux Informatiques
15h30-17h00	Repos	Programmation Java	SQL Server	English	Programmation Linéaire

Groupe 8
2ème étage | Salle 108 | Sans conflit

	Lundi	Mardi	Mercredi	Jeudi	Vendredi
08h30-10h00	Repos	SQL Server	Repos	Repos	Réseaux Informatiques
10h15-11h45	Programmation Linéaire	Programmation Python	English	Repos	Repos
12h00-13h30	Repos	Programmation Java	Repos	Repos	Repos
13h45-15h15	Recherche Scientifique	Développement Projet	Modèles Statistiques	Repos	Modèles Statistiques
15h30-17h00	Programmation Java	Communication Pro	Conception OO	Repos	Repos

Groupe 9
2ème étage | Salle 109 | Sans conflit

	Lundi	Mardi	Mercredi	Jeudi	Vendredi
08h30-10h00	Repos	Développement Projet	Repos	Repos	SQL Server
10h15-11h45	English	Repos	Repos	Programmation Java	Modèles Statistiques
12h00-13h30	Repos	Recherche Scientifique	Programmation Java	Repos	Repos
13h45-15h15	Programmation Linéaire	Réseaux Informatiques	Repos	Programmation Python	Repos
15h30-17h00	Conception OO	Repos	Repos	Modèles Statistiques	Communication Pro

Groupe 10
2ème étage | Salle 110 | Sans conflit

	Lundi	Mardi	Mercredi	Jeudi	Vendredi
08h30-10h00	Développement Projet	Programmation Java	Modèles Statistiques	Repos	Repos
10h15-11h45	Communication Pro	Conception OO	Réseaux Informatiques	English	Modèles Statistiques
12h00-13h30	Recherche Scientifique	Repos	Repos	Programmation Java	Repos
13h45-15h15	Repos	Repos	Repos	SQL Server	Repos
15h30-17h00	Repos	Programmation Linéaire	Repos	Repos	Programmation Python

Le code :

```
import streamlit as st
from scipy.optimize import linprog
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
```

```

import random

import pandas as pd

import base64

from pathlib import Path

from collections import defaultdict


# -----
# ENCODE LOGO EN BASE64
# -----

def get_logo_base64():
    parent = Path(__file__).parent
    candidates = [
        "emsi_logo.jpg",
        "emsi_logo (1).jpg",
        "OIP.jpg",
        "logo.jpg",
        "logo.png",
    ]
    for name in candidates:
        p = parent / name
        if p.exists():
            with open(p, "rb") as f:
                return base64.b64encode(f.read()).decode()
    return None


# -----
# STYLE CSS - THEME EMSI VERT
# -----

def load_css(logo_b64):
    logo_html = (

```

```

f''

if logo_b64 else

'<div style="font-family:Montserrat,sans-serif;font-size:1.6rem;font-weight:900;color:#2E7D32;letter-
spacing:3px;">EMSI</div>'

)

st.markdown(f"""

<style>

@import
url('https://fonts.googleapis.com/css2?family=Montserrat:wght@400;600;700;800;900&family=Roboto:wg
ht@400;500;600&display=swap');

html, body, [class*="css"] {{ font-family: 'Roboto', sans-serif; background-color: #F4F6F4; }}

.main {{ background-color: #F4F6F4; }}

.emsi-header {{

    background: linear-gradient(135deg, #1B5E20 0%, #2E7D32 55%, #388E3C 100%);

    border-radius: 18px; padding: 24px 36px; margin-bottom: 28px;

    display: flex; align-items: center; gap: 28px;

    box-shadow: 0 8px 32px rgba(27,94,32,0.30);

}}

.emsi-logo-box {{

    background: white; border-radius: 14px; padding: 12px 18px;

    display: flex; align-items: center; justify-content: center;

    box-shadow: 0 4px 16px rgba(0,0,0,0.15); min-width: 150px;

}}

.emsi-header-text h1 {{

    font-family: 'Montserrat', sans-serif; font-size: 1.9rem; font-weight: 800;

    color: #FFFFFF !important; margin: 0 0 6px 0;

}}

.emsi-header-text p {{ color: rgba(255,255,255,0.88); font-size: 0.92rem; margin: 0 0 8px 0; }}

.emsi-badge {{

    background: rgba(255,255,255,0.20); border: 1px solid rgba(255,255,255,0.40);

    border-radius: 20px; padding: 4px 14px; font-size: 0.78rem; color: #fff;

```

```

    display: inline-block; font-weight: 600; margin-right: 8px;
  }}
h2, h3 {{ font-family: 'Montserrat', sans-serif; color: #1B5E20; }}
[data-testid="metric-container"] {{
  background: #FFFFFF; border-radius: 14px; padding: 18px; text-align: center;
  border-top: 4px solid #2E7D32; box-shadow: 0 4px 18px rgba(46,125,50,0.12);
}}
[data-testid="metric-container"] label {{ color: #555; font-size: 0.85rem; font-weight: 600; }}
[data-testid="metric-container"] [data-testid="stMetricValue"] {{
  color: #2E7D32; font-family: 'Montserrat', sans-serif; font-size: 1.9rem; font-weight: 800;
}}
.stButton>button {{
  background: linear-gradient(135deg, #1B5E20 0%, #2E7D32 100%);
  color: white; border-radius: 12px; height: 3.3em; width: 100%;
  font-family: 'Montserrat', sans-serif; font-weight: 700; font-size: 1rem;
  border: none; box-shadow: 0 4px 16px rgba(46,125,50,0.35); transition: all 0.2s ease;
}}
.stButton>button:hover {{ background: linear-gradient(135deg, #2E7D32 0%, #43A047 100%);
transform: translateY(-2px); }}
[data-testid="stSidebar"] {{ background: linear-gradient(180deg, #0A1F0B 0%, #1B3D1C 100%); }}
[data-testid="stSidebar"] .stMarkdown,
[data-testid="stSidebar"] label,
[data-testid="stSidebar"] p,
[data-testid="stSidebar"] div {{ color: #E8F5E9 !important; }}
.sidebar-logo-zone {{
  background: rgba(255,255,255,0.08); border: 1px solid rgba(255,255,255,0.18);
  border-radius: 14px; padding: 16px; text-align: center; margin-bottom: 20px;
}}
.stProgress > div > div {{ background: linear-gradient(90deg, #1B5E20, #66BB6A); border-radius: 8px; }}
.stProgress > div {{ background: #C8E6C9; border-radius: 8px; }}
.info-card {{

```

```

        background: #FFFFFF; border-radius: 12px; padding: 16px 22px; margin: 10px 0;
        border-left: 5px solid #2E7D32; box-shadow: 0 2px 12px rgba(46,125,50,0.10);
    }}
    .conflict-card {{
        background: #FFF3E0; border-radius: 12px; padding: 16px 22px; margin: 10px 0;
        border-left: 5px solid #E65100; box-shadow: 0 2px 12px rgba(230,81,0,0.15);
    }}
    .ok-card {{
        background: #F1F8E9; border-radius: 12px; padding: 16px 22px; margin: 10px 0;
        border-left: 5px solid #2E7D32; box-shadow: 0 2px 12px rgba(46,125,50,0.10);
    }}
    hr {{ border: none; height: 2px; background: linear-gradient(90deg, #2E7D32, transparent); margin:
24px 0; }}
</style>

```

```

<div class="emsi-header">
    <div class="emsi-logo-box">{logo_html}</div>
    <div class="emsi-header-text">
        <h1>  EMSI Room Optimizer</h1>
        <p>Dashboard intelligent de répartition optimale des groupes et des salles</p>
        <span class="emsi-badge">  EMSI Marrakech</span>
        <span class="emsi-badge">  Programmation Linéaire</span>
        <span class="emsi-badge">  Anti-Conflicts</span>
    </div>
</div>

```

```

"""', unsafe_allow_html=True)

```

```

# =====
# CLASSES POO
# =====

class Solution:

```

```
def __init__(self, x, y, z, valeur):
    self.x = x; self.y = y; self.z = z; self.valeur = valeur
```

```
class ProblemeOptimisation:
```

```
def __init__(self, total_groupes, heures):
    self.total_groupes = total_groupes; self.heures = heures
```

```
class Solveur:
```

```
def resoudre(self, p):
    c = [-1, -1]
    A = [[1, 1], [p.heures, 0], [0, p.heures], [1, -1]]
    b = [p.total_groupes, 175, 175, 4]
    result = linprog(c, A_ub=A, b_ub=b, bounds=(0, None))
    x, y = result.x if result.success else (0, 0)
    z = max(0, p.total_groupes - x - y)
    return Solution(x, y, z, x + y)
```

```
# =====
```

```
# CONSTANTES
```

```
# =====
```

```
# Matières passant 1 fois/semaine
```

```
MATIERES_1X = [
    "Programmation Python", "SQL Server",
    "Développement Projet", "Programmation Linéaire", "Réseaux Informatiques",
    "Recherche Scientifique", "Communication Pro", "English",
    "Conception OO",
]
```

```
# Matières passant 2 fois/semaine
```

```
MATIERES_2X = ["Modèles Statistiques", "Programmation Java"]
```

```
JOURS = ["Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi"]
```



```
CRENEAUX = ["08h30-10h00", "10h15-11h45", "12h00-13h30", "13h45-15h15", "15h30-17h00"]
```

```
# Grille semaine : 5 jours x 5 créneaux = 25 cases
```

```
# Plan fixe par groupe :
```

```
# 9 matières 1x = 9 cases
```

```
# 2 matières 2x = 4 cases
```

```
# Total cours = 13 cases
```

```
# Repos = 12 cases (pauses déjeuner, récupération)
```

```
def construire_plan_semaine() -> list:
```

```
    """
```

```
    Retourne une liste de 25 éléments (5 jours x 5 créneaux) représentant  
    les matières à planifier pour une semaine, avec repos.
```

```
    """
```

```
    plan = (
```

```
        MATIERES_1X[:]          + # 9 matières × 1
```

```
        MATIERES_2X * 2          + # 2 matières × 2 = 4
```

```
        ["🌿 Repos"] * 12        # 12 créneaux de repos
```

```
    )
```

```
    assert len(plan) == 25, f"Plan invalide : {len(plan)} cases"
```

```
    random.shuffle(plan)
```

```
    return plan
```

```
# =====
```

```
# AFFECTATION DES SALLES UNIQUE PAR GROUPE
```

```
# =====
```

```
def affecter_salles_uniques(nb_groupes_total: int) -> dict:
```

```
    """
```

```
    Retourne un dict {groupe_id: salle_unique} où chaque groupe
```

```
    a une salle différente. Les salles sont nommées globalement
```

```
    (Salle 101, 102 ... 115 ...) pour éviter tout doublon entre étages.
```

Règle :

- 1er étage : Salles 101-110 (max 10 salles)
- 2ème étage : Salles 201-210
- 3ème étage : Salles 301-310

On attribue les salles séquentiellement → jamais deux groupes dans la même salle.

"""

affectation = {}

Pool de salles par étage (suffisamment grand)

pool = (

["Salle 1{i:02d}" for i in range(1, 21)] + # étage 1 : 101-120

["Salle 2{i:02d}" for i in range(1, 21)] + # étage 2 : 201-220

["Salle 3{i:02d}" for i in range(1, 21)] # étage 3 : 301-320

)

for gid in range(1, nb_groupes_total + 1):

 affectation[gid] = pool[gid - 1] # index unique → salle unique

return affectation

=====

GESTIONNAIRE EDT (UNE SALLE FIXE PAR GROUPE)

=====

class GestionnaireEDT:

"""

Règles garanties :

☒ Pas de conflit de GROUPE : un groupe a 1 seul cours par créneau.

☒ Pas de conflit de SALLE : deux groupes ne partagent JAMAIS
la même salle (affectation unique).

☒ Variété des matières : pas deux fois la même matière le même jour.

"""

def __init__(self):

```

# grille globale : (salle, jour, creneau) -> groupe_id [contrôle salle]
self.grille_salle = {}

# grille globale : (groupe_id, jour, creneau) -> matiere [contrôle groupe]
self.grille_groupe = {}

self.edts = {} # groupe_id -> (DataFrame, salle)
self.conflicts_log = []

def generer_edt_groupe(self, groupe_id: int, salle: str):
    """
    Génère l'EDT d'un groupe avec les règles :

    - Chaque matière 1x apparaît exactement 1 fois dans la semaine
    - Modèles Statistiques et Programmation Java apparaissent 2 fois
    - Les cases restantes sont des créneaux de repos
    - Pas de conflit de salle ni de groupe
    """

    # Construire le plan de la semaine (25 cases mélangées)
    plan = construire_plan_semaine()
    cases = [(j, c) for j in JOURS for c in CRENEAUX] # 25 cases ordonnées

    edt = {}

    for idx, (jour, creneau) in enumerate(cases):
        cle_salle = (salle, jour, creneau)
        cle_groupe = (groupe_id, jour, creneau)
        matiere = plan[idx]

        # — Vérification conflit de salle (sécurité) —————
        if cle_salle in self.grille_salle:
            occupant = self.grille_salle[cle_salle]
            self.conflicts_log.append(
                f"⚡ Conflit SALLE : **{salle}** le **{jour}** à **{creneau}** "
            )

```

```

        f"déjà prise par Groupe {occupant} → Groupe {groupe_id} bloqué"
    )

    edt[(jour, creneau)] = "⊖ CONFLIT SALLE"

    continue

# — Vérification conflit de groupe (sécurité) —————
if cle_groupe in self.grille_groupe:
    self.conflits_log.append(
        f"⊗ Conflit GROUPE : **Groupe {groupe_id}** déjà planifié "
        f"le **{jour}** à **{creneau}**"
    )

    edt[(jour, creneau)] = "⊖ CONFLIT GROUPE"

    continue

# — Enregistrer (pas de repos dans les grilles de contrôle) —
if matiere != "🛑 Repos":
    self.grille_salle[cle_salle] = groupe_id
    self.grille_groupe[cle_groupe] = matiere

    edt[(jour, creneau)] = matiere

# Construire DataFrame : index = créneaux, colonnes = jours
data = {jour: [edt.get((jour, c), "—") for c in CRENEAUX] for jour in JOURS}
df = pd.DataFrame(data, index=CRENEAUX)
self.edts[groupe_id] = (df, salle)

return df

# — Vérification globale a posteriori —————
def rapport_conflits(self):
    """

    Relit toutes les grilles et retourne :

```

```

    nb_conflits_salle, nb_conflits_groupe, liste_details
"""

# Conflit salle : même (salle, jour, creneau) → 2 groupes
inv_salle = defaultdict(list)

for (salle, jour, cren), gid in self.grille_salle.items():
    inv_salle[(salle, jour, cren)].append(gid)

details = []
nb_cs = nb_cg = 0
for (salle, jour, cren), gids in inv_salle.items():
    if len(gids) > 1:
        nb_cs += 1
        details.append(
            f"● Conflit SALLE : **{salle}** le **{jour}** à **{cren}** "
            f"→ Groupes {gids}"
        )

# Conflit groupe : même (groupe_id, jour, creneau) → 2 matières
inv_groupe = defaultdict(list)
for (gid, jour, cren), mat in self.grille_groupe.items():
    inv_groupe[(gid, jour, cren)].append(mat)
for (gid, jour, cren), mats in inv_groupe.items():
    if len(mats) > 1:
        nb_cg += 1
        details.append(
            f"● Conflit GROUPE : **Groupe {gid}** le **{jour}** à **{cren}** "
            f"→ Matières {mats}"
        )

# Ajouter les conflits détectés lors de la génération
details += self.conflits_log

```

```

nb_cs += sum(1 for l in self.conflits_log if "SALLE" in l)
nb_cg += sum(1 for l in self.conflits_log if "GROUPE" in l)
return nb_cs, nb_cg, details

```

```

# =====

```

```

# CONFIG PAGE

```

```

# =====

```

```

st.set_page_config(page_title="EMSI Marrakech — Optimizer", page_icon="🎓", layout="wide")

```

```

logo_b64 = get_logo_base64()

```

```

load_css(logo_b64)

```

```

# =====

```

```

# SIDEBAR

```

```

# =====

```

```

with st.sidebar:

```

```

    if logo_b64:

```

```

        st.markdown(f"""

```

```

            <div class="sidebar-logo-zone">

```

```

```

```

                <div style="margin-top:10px;font-size:0.75rem;color:rgba(255,255,255,0.7);line-height:1.6;">

```

```

                    École Marocaine des Sciences de l'Ingénieur<br>

```

```

                    <b style="color:#81C784;"> 📍 Marrakech</b>

```

```

                </div>

```

```

            </div>

```

```

        """, unsafe_allow_html=True)

```

```

st.markdown("### ⚙️ Configuration")

```

```

total_groupes = st.slider("Nombre de groupes", 1, 30, 12)

```

```

heures = st.slider("Heures / groupe", 1, 30, 21)

```

```

st.markdown("---")
st.markdown("""
    <div style="color:rgba(200,230,200,0.85);font-size:0.82rem;line-height:1.9;">
        🎓 <b style="color:#81C784;">EMSI Marrakech</b><br>
        💡 Optimisation des salles<br>
        🏢 Génie Informatique<br>
        🛡️ Anti-conflits activé<br>
        🤝 Membre HONORES United Universities
    </div>
    """, unsafe_allow_html=True)

```

```

# =====
# BOUTON PRINCIPAL
# =====

if st.button("🔍 Lancer l'optimisation"):

```

```

# — Résolution LP —————
p = ProblemeOptimisation(total_groupes, heures)
solver = Solveur()
sol = solver.resoudre(p)
st.success("✅ Optimisation réussie — Résultats affichés ci-dessous")

```

```

# — KPIs —————
col1, col2, col3, col4 = st.columns(4)
col1.metric("🏠 1er étage", f"{sol.x:.2f}")
col2.metric("🏠 2ème étage", f"{sol.y:.2f}")
col3.metric("🏠 3ème étage", f"{sol.z:.2f}")
col4.metric("📊 Performance Z", f"{sol.valeur:.2f}")

st.markdown("---")

```

— Graphique

```
st.subheader("📊 Visualisation de la répartition")

fig, ax = plt.subplots(figsize=(8, 4))

fig.patch.set_facecolor('FFFFFF'); ax.set_facecolor('#F1F8E9')

bars = ax.bar(["1er étage", "2ème étage", "3ème étage"],
              [sol.x, sol.y, sol.z],
              color=['#1B5E20', '#388E3C', '#81C784'],
              width=0.5, edgecolor='white', linewidth=2)

for bar, val in zip(bars, [sol.x, sol.y, sol.z]):
    ax.text(bar.get_x()+bar.get_width()/2., bar.get_height()+0.08,
            f'{val:.1f}', ha='center', va='bottom',
            color='#1B5E20', fontweight='bold', fontsize=12)

ax.tick_params(colors='#333333', labelsiz=11)

ax.set_ylabel("Nombre de Groupes", color='#333333', fontsize=11)

ax.set_title("Distribution optimale des salles — EMSI Marrakech",
            color='#1B5E20', fontsize=13, fontweight='bold', pad=14)

for sp in ax.spines.values(): sp.set_color('#C8E6C9')

ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)

ax.yaxis.grid(True, color='#C8E6C9', linestyle='--', alpha=0.8)

ax.set_axisbelow(True)

st.pyplot(fig)
```

— Insight + taux

```
st.subheader("🤖 Analyse automatique")

insight = ("Le 1er étage est prioritaire." if sol.x > sol.y else
          "Le 2ème étage est prioritaire." if sol.y > sol.x else
          "Répartition équilibrée entre les étages.")

st.markdown(f'<div class="info-card">🔍 <b>Insight :</b> {insight}</div>',
            unsafe_allow_html=True)
```



```

taux = min(100, int((sol.valeur / total_groupes) * 100))

st.subheader("📊 Taux d'utilisation globale")

st.progress(taux)

st.markdown(f'<div class="info-card">📊 <b>Utilisation :</b> '
            f'<span style="color:#2E7D32;font-size:1.15rem;font-weight:800;">{taux}%</span>'
            f'</div>', unsafe_allow_html=True)

```

```

# =====

```

```

# GÉNÉRATION DES EDT — SALLES UNIQUES PAR GROUPE

```

```

# =====

```

```

st.markdown("---")

```

```

st.subheader("📅 Génération des emplois du temps — Détection de conflits")

```

```

nb_e1 = int(sol.x)

```

```

nb_e2 = int(sol.y)

```

```

nb_e3 = int(sol.z)

```

```

nb_total = nb_e1 + nb_e2 + nb_e3

```

```

# — Attribution salles uniques globales —————

```

```

affectation = affecter_salles_uniques(nb_total)

```

```

# Déterminer étage de chaque groupe

```

```

etages = {}

```

```

for gid in range(1, nb_e1 + 1):

```

```

    etages[gid] = "1er étage"

```

```

for gid in range(nb_e1 + 1, nb_e1 + nb_e2 + 1):

```

```

    etages[gid] = "2ème étage"

```

```

for gid in range(nb_e1 + nb_e2 + 1, nb_total + 1):

```

```

    etages[gid] = "3ème étage"

```

```

# — Génération des EDT —————

```

```

gestionnaire = GestionnaireEDT()

for gid in range(1, nb_total + 1):
    salle = affectation[gid]
    gestionnaire.generer_edt_groupe(gid, salle)

# — Rapport de conflits —————

nb_cs, nb_cg, details = gestionnaire.rapport_conflits()
total_conflits = nb_cs + nb_cg

st.markdown("### 📋 Rapport de Conflits")
cA, cB, cC = st.columns(3)

cA.metric("⚠️ Conflits de Salle", nb_cs)
cB.metric("⚠️ Conflits de Groupe", nb_cg)
cC.metric("✅ Total Conflits", total_conflits)

if total_conflits == 0:
    st.markdown(
        '<div class="ok-card">'
        '✅ <b>Aucun conflit détecté</b> — Chaque groupe a sa propre salle unique. '
        'Tous les emplois du temps sont cohérents.'
        '</div>', unsafe_allow_html=True
    )
else:
    st.markdown(
        f'<div class="conflict-card">'
        f'🚫 <b>{total_conflits} conflit(s) détecté(s)</b> — '
        f'{nb_cs} salle(s), {nb_cg} groupe(s).'
        f'</div>', unsafe_allow_html=True
    )

with st.expander("📄 Voir le détail des conflits"):
    for d in details:

```

```

st.markdown(d)

# — Résumé des affectations —————
st.markdown("---")
st.subheader("🏠 Affectation des salles")

resume_data = {
    "Groupe": [f"Groupe {gid}" for gid in range(1, nb_total + 1)],
    "Étage": [etages[gid] for gid in range(1, nb_total + 1)],
    "Salle": [affectation[gid] for gid in range(1, nb_total + 1)],
}

df_resume = pd.DataFrame(resume_data)
st.dataframe(df_resume, width='stretch', hide_index=True)

# — EDT détaillés par groupe —————
st.markdown("---")
st.subheader("📅 Emplois du temps détaillés")

def style_edt(val):
    v = str(val)
    if "CONFLIT" in v:
        return "background-color:#FFCDD2; color:#B71C1C; font-weight:bold;"
    if "Repos" in v:
        return "background-color:#E8F5E9; color:#757575; font-style:italic;"
    if "Statistiques" in v or "Java" in v:
        return "background-color:#FFF9C4; color:#5D4037; font-weight:bold;"
    return "background-color:#F1F8E9; color:#1B5E20;"

for gid in range(1, nb_total + 1):
    salle = affectation[gid]
    etage = etages[gid]

```

```

df, _ = gestionnaire.edts[gid]

has_conflict = any("CONFLIT" in str(v) for v in df.values.flatten())

st.markdown(
    f'<div class="{ "conflict-card" if has_conflict else "info-card" }">'
    f'<b style="font-size:1.05rem;color:#1B5E20;">📅 Groupe {gid}</b><br>'
    f'🏢 {etage} &nbsp;|&nbsp; 🚪 <b>{salle}</b>'
    f'{"&nbsp;&nbsp;&nbsp;⚠️ <span style=color:#E65100;>Conflit détecté</span>" if has_conflict else
"&nbsp;&nbsp;&nbsp;✅ <span style=color:#2E7D32;>Sans conflit</span>"}'
    f'</div>', unsafe_allow_html=True
)

try:
    styled = df.style.map(style_edt)
except AttributeError:
    styled = df.style.applymap(style_edt)

st.dataframe(styled, width='stretch')

st.markdown("---")

```

III.4. Conclusion

La partie informatique de ce projet a permis de transformer le modèle mathématique d'optimisation en une application fonctionnelle et interactive développée en Python. Grâce à l'utilisation de la programmation orientée objet, le système a été structuré de manière modulaire, facilitant ainsi la compréhension, la maintenance et l'évolution du code. L'intégration de bibliothèques spécialisées telles que SciPy, Streamlit, Pandas et Matplotlib a permis d'assurer à la fois la résolution du problème d'optimisation, la gestion des données et la visualisation des résultats. Par ailleurs, le développement d'une interface utilisateur intuitive a rendu l'application accessible et exploitable de manière simple par l'utilisateur final. Ainsi, cette partie confirme l'efficacité de l'approche adoptée pour automatiser la prise de décision dans la gestion des salles et des emplois du temps.

CONCLUSION GENERALE ET PERSPECTIVES

Ce projet a permis de concevoir et de développer une solution complète d'optimisation de la répartition des salles pour les groupes de 3^e année Génie Informatique à l'EMSI. En s'appuyant sur la programmation linéaire et la méthode du simplexe, le problème a été modélisé mathématiquement puis résolu de manière

automatique à l'aide de Python et de ses bibliothèques scientifiques. L'implémentation a été réalisée selon une approche orientée objet, garantissant une meilleure structuration du code, une modularité accrue et une facilité de maintenance.

Par ailleurs, l'intégration d'une interface interactive développée avec Streamlit a permis de transformer la solution en une application conviviale et accessible, offrant à l'utilisateur une visualisation claire des résultats sous forme de tableaux, indicateurs et graphiques. Le système développé assure également la génération automatique des emplois du temps tout en intégrant des mécanismes de détection et de prévention des conflits, ce qui améliore la fiabilité de la planification.

En perspective, cette application peut être enrichie par l'intégration d'algorithmes d'optimisation plus avancés, tels que les méthodes heuristiques ou métaheuristiques, afin de traiter des problèmes de plus grande complexité. Il serait également intéressant d'ajouter une base de données pour la gestion persistante des emplois du temps ainsi qu'un système multi-utilisateurs accessible en réseau. Enfin, l'amélioration de l'interface graphique et l'ajout de fonctionnalités d'intelligence artificielle pourraient permettre d'optimiser davantage les décisions et d'adapter automatiquement la répartition en fonction de nouvelles contraintes.

ANNEXES



Matières : Programmation Linéaire / Python

Cahier des charges et Charte du Projet

Projet : ...Optimisation des salles du 1^{er} étage, 2^{ème} étage et 3^{ème} étage dans Emsi Gueliz pour 3^{ème} année genie informatique et réseau

1. Contexte et Objectifs du Projet

Ce projet s'inscrit dans un contexte marqué par un besoin croissant d'optimisation, de modernisation et d'amélioration des performances au sein de l'organisation, confrontée à des contraintes techniques, organisationnelles et concurrentielles de plus en plus exigeantes. Face à l'évolution des technologies, à l'augmentation des attentes des utilisateurs et à la nécessité d'améliorer la qualité, la fiabilité et la rapidité des services proposés, il devient indispensable de mettre en place une solution structurée, efficace et durable. L'objectif principal du projet est donc de concevoir et de déployer une solution adaptée permettant de répondre précisément aux besoins identifiés, tout en respectant les contraintes de coût, de délai et de qualité définies dans le cahier des charges. Plus spécifiquement, le projet vise à analyser la situation existante, identifier les dysfonctionnements ou limites actuelles, proposer une solution technique et/ou organisationnelle optimisée, améliorer la performance globale du système, renforcer la satisfaction des utilisateurs finaux et assurer une meilleure gestion des ressources. À travers cette démarche, l'organisation ambitionne également de garantir la pérennité de ses activités, d'augmenter sa compétitivité et de s'inscrire dans une dynamique d'amélioration continue. Le présent projet porte sur la conception et la

modélisation d'un système d'optimisation des ressources académiques pour les classes de 3^{ème} année en Ingénierie Informatique à l'EMSI Gueliz. L'enjeu central est de résoudre le problème complexe de l'ordonnancement des cours (Timetabling) en utilisant les principes de la programmation linéaire. L'objectif principal est de maximiser l'efficacité de l'emploi du temps en assurant une répartition fluide des séances sur les trois niveaux dédiés du campus (Étages 1, 2 et 3), tout en minimisant les périodes d'inactivité pour les étudiants et en optimisant l'usage des salles de classe.

Pour atteindre cet objectif, le modèle intègre une série de contraintes critiques garantissant la viabilité de la solution. Parmi elles, les contraintes d'unicité empêchent tout conflit d'horaire pour les professeurs et les groupes d'étudiants, tandis que les contraintes de capacité veillent à ce que chaque groupe soit affecté à une salle adaptée à son effectif. Une attention particulière est portée à la dimension géographique du projet : en limitant les variables aux étages 1, 2 et 3, le modèle cherche à réduire les déplacements inutiles et à saturer intelligemment les espaces disponibles avant de solliciter d'autres ressources.

Sur le plan technique, ce problème est formulé comme un programme linéaire en variables mixtes. La fonction objectif vise à maximiser un score de "confort" et d'efficacité, pondéré par le respect des volumes horaires pédagogiques. Ce projet ne se contente pas d'une approche théorique ; il utilise des outils de résolution performants comme la bibliothèque PuLP ou Google OR-Tools sous Python pour transformer des contraintes logistiques réelles en une solution mathématique rigoureuse, offrant ainsi une gestion moderne et automatisée de la vie scolaire à l'EMSI.

2. Périmètre et Déroulement du Projet

Le projet est structuré en trois parties.

- **Partie 1 : Modélisation et Résolution par la Méthode Graphique**

Dates de remise : entre le 04/05/2026 et le 09/05/2026

4. Contraintes et Exigences

- Le rapport doit strictement respecter le modèle fourni par les encadrants.
- Les solutions proposées doivent être interprétées de manière claire et compréhensible, même pour un public non spécialiste.
- Le rapport final doit comporter une analyse détaillée des résultats et formuler des recommandations pertinentes.
- Les annexes doivent inclure l'ensemble des programmes informatiques développés.
- Toutes les figures et schémas doivent être correctement commentés.

5. Organisation et Planning Prévisionnel

- Phase 1 : Modélisation et résolution graphique — Semaines 1 à 4
- Phase 2 : Modélisation et résolution Simplexe/Dualité — Semaines 5 à 8 → Livrable 1
- Phase 3 : Développement informatique — Semaines 9 à 12 → Livrable 2

6. Ressources Nécessaires

- Outils de modélisation et de test : Python, Excel, ou autres plateformes équivalentes.
- Documentation sur les méthodes d'optimisation (méthode graphique, Simplexe, dualité).

7. Conclusion

Ce projet vise à illustrer l'importance des méthodes d'optimisation dans l'amélioration des processus. Il offre l'opportunité d'appliquer des outils mathématiques à un cas concret et d'en analyser les impacts opérationnels.

Les résultats attendus permettront une meilleure gestion des ressources et contribueront à une prise de décision plus éclairée.

Signature du chef de projet	Signature du Pr A.REHA	Signature du Pr
.....		

Membres de l'équipe :

~~Aghamri~~ badreddine

Talami Abderrahman.....[.....](#)

~~Malainine~~ Mohamed.....[.....](#)

REFERENCES

- [1] K. F. Lee, K. M. Luk, et H. W. Lai, *Microstrip patch antennas*, Second edition. Singapore ; New Jersey: World Scientific Publishing Co. Pte. Ltd, 2018.
- [2] UNet, *Advances in ubiquitous networking 2: proceedings of the UNet'16*. in Lecture notes in electrical engineering, no. Volume 397. Singapore: Springer, 2017.